ĐẶC TẢ THIẾT KẾ gemm_register_file

Module của `gemm_accel_top` · instance `u_regfile`

Mục lục

1 Tổng quan **2**

1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2
1.4 Tín hiệu AXI . . . 4

2 Đặc tả thanh ghi **5**

3 Mô tả chức năng **7**

3.1 Tổ chức chức năng . . . 8
3.2 Thanh ghi . . . 8
3.3 Timing . . . 9
3.4 Reset, hiện thực, kiểm chứng . . . 10

1 Tổng quan

1.1 Tính năng chính

Register File là đầu mối duy nhất giữa CPU và accelerator. Nó trả lời mọi truy cập MMIO trên AXI4-Lite, lưu DESC_PTR, quyết định chấp nhận START, chụp DESC_PTR đúng lúc chấp nhận, giữ cờ trạng thái DONE/ERROR, và đếm chu kỳ hiệu năng. Nó không biết gì về các pha: Main FSM báo lại khi tác vụ xong. Module nằm trong miền clk duy nhất (100 MHz, chu kỳ 10,0 ns) và nhận rst_n_sync từ top.

Chức năng

- Mô hình lập trình chỉ có một chế độ: chuỗi descriptor. CPU ghi địa chỉ descriptor đầu chuỗi vào DESC_PTR, rồi ghi CTRL[2] = START. Mọi tham số của từng job (kích thước, địa chỉ, stride, cờ, lượng tử) nằm trong descriptor ở DRAM, không có thanh ghi MMIO cho chúng
- Register map và bố trí bit nằm ở mục Đặc tả thanh ghi bên dưới; module này hiện thực giải mã, ghi byte, đọc lại và cập nhật trạng thái của giao diện đó
- Chỉ nhận START khi rảnh. Chụp DESC_PTR và báo start trước khi hoàn tất phản hồi ghi AXI. DONE/ERROR giữ đến tác vụ mới hoặc reset; ERR_CODE bằng 0 khi không lỗi
- Chụp DESC_PTR khi nhận tác vụ; giữ nguyên tới lần chấp nhận kế tiếp
- Các bộ đếm PERF xóa khi nhận tác vụ, đếm khi BUSY, bão hòa tại 0xFFFF_FFFF và giữ sau kết thúc. Khi các pha chồng nhau, tổng các bộ đếm pha lớn hơn tổng thời gian tác vụ
- Luôn trả OKAY; không bao giờ treo bus. Offset dự trữ đọc 0, ghi bị bỏ qua

> **QUYẾT ĐỊNH BASELINE D-01**
> Baseline chỉ có chế độ chuỗi descriptor, theo spec_1. Offset 0x08 … 0x24 là dự trữ (đọc 0, ghi bị bỏ qua); CTRL[0] dự trữ; STATUS[3] dự trữ, đọc 0. Giao diện Register File → FSM chỉ có ba tín hiệu: o_start_pulse, o_soft_req, o_cfg_desc_ptr; không có o_start_kind, không có bản chụp kích thước/địa chỉ/hậu xử lý. Lý do: kiến trúc chốt là DMA theo descriptor; chế độ lập trình bằng thanh ghi là nhánh thiết kế thứ hai, trùng chức năng.

1.2 Tham số

Bảng 1.1: Tham số

Tham số	Mặc định	Ý nghĩa
P	8	Ghi vào HW_CONFIG[7:0]. Giá trị hợp lệ duy nhất là 8; giá trị khác bị chặn lúc elaboration ở top

> **QUYẾT ĐỊNH BASELINE D-04**
> P = 8 cố định (array 8 × 8 PE). Register File chỉ dùng P làm hằng ghi vào HW_CONFIG.

1.3 Tín hiệu giao tiếp

Bảng 1.2: Tín hiệu giao tiếp của gemm_register_file

Tên tín hiệu	Độ rộng	Hướng	Mô tả
clk	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.

Bảng 1.2: Tín hiệu giao tiếp của `gemm_register_file` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module (nối vào `rst_n_sync`).
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Register File → FSM			
`o_start_pulse`	1	Output	Xung một chu kỳ báo Register File vừa chấp nhận một tác vụ. `DESC_PTR` đã chụp hợp lệ cùng chu kỳ.
`o_soft_req`	1	Output	Xung một chu kỳ yêu cầu Main FSM thực hiện soft reset. Phát khi lệnh ghi CTRL có SOFT_RESET = 1 và byte 0 được cho phép ghi.
`o_cfg_desc_ptr`	32	Output	Bản chụp đủ 32 bit của DESC_PTR. Địa chỉ descriptor đầu chuỗi. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
FSM → Register File			
`i_soft_busy`	1	Input	Báo đang xử lý soft reset. Trong khoảng này Register File báo BUSY = 1 và không nhận tác vụ mới.
`i_done_pulse`	1	Input	Xung một chu kỳ báo toàn bộ tác vụ kết thúc thành công. Register File đặt DONE và xóa BUSY tại cạnh lên; không xuất hiện cùng err pulse.
`i_err_pulse`	1	Input	Xung một chu kỳ báo toàn bộ tác vụ kết thúc với lỗi. Mã lỗi hợp lệ cùng chu kỳ; Register File đặt ERROR và xóa BUSY tại cạnh lên.
`i_err_code`	4	Input	Mã lỗi của tác vụ, được lấy mẫu cùng err pulse. Giá trị 1: cấu hình. Giá trị 2: đọc dữ liệu. Giá trị 3: ghi kết quả; 4: đọc descriptor. Giá trị 5 … 15 dự trữ, không bao giờ xuất hiện.
`i_perf_load`	1	Input	Báo pha LOAD dữ liệu, không bao gồm FETCH đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_perf_comp`	1	Input	Báo pha COMPUTE đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_perf_store`	1	Input	Báo pha STORE_C đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_err_desc_addr`	32	Input	Địa chỉ descriptor lỗi được cung cấp cho thanh ghi ERR_DESC_ADDR tại offset 0x44.
`i_desc_done_cnt`	32	Input	Số descriptor đã hoàn tất STORE_C thành công, cung cấp cho thanh ghi DESC_DONE_CNT tại offset 0x48.

Giao tiếp

- Mọi port ra (`s_axi_*` ra và `o_*`) lấy từ thanh ghi, trừ `s_axi_awready`, `s_axi_wready`, `s_axi_arready`: ba tín hiệu này là hàm tổ hợp của cờ trạng thái nội bộ (thanh ghi), không phụ thuộc tổ hợp vào input
- `o_cfg_desc_ptr` chỉ đổi ở cạnh clock chấp nhận tác vụ; ổn định suốt tác vụ
- `i_done_pulse` và `i_err_pulse` chỉ đến khi BUSY = 1 và không bao giờ cùng chu kỳ
- `o_start_pulse` và `o_soft_req` không bao giờ cùng chu kỳ (SOFT_RESET thắng START trong cùng một lệnh ghi)
- Giao diện Register File → FSM gồm đúng ba tín hiệu trên; không có đường nào khác mang cấu hình tới FSM

1.4 Tín hiệu AXI

Bảng 1.3: Tín hiệu AXI

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`s_axi_awaddr`	32	Input	Địa chỉ byte của thanh ghi cần ghi. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
`s_axi_awvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
`s_axi_awready`	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
`s_axi_wdata`	32	Input	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
`s_axi_wstrb`	4	Input	Cho phép ghi từng byte của thanh ghi RW. Bit b = 1 cho phép ghi byte b; bit b = 0 giữ byte đó. Các bit lệnh CTRL chỉ có hiệu lực khi bit 0 bằng 1.
`s_axi_wvalid`	1	Input	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
`s_axi_wready`	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
`s_axi_bresp`	2	Output	Mã phản hồi ghi. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng BVALID.
`s_axi_bvalid`	1	Output	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
`s_axi_bready`	1	Input	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1.
`s_axi_araddr`	32	Input	Địa chỉ byte của thanh ghi cần đọc. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
`s_axi_arvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.

Bảng 1.3: Tín hiệu AXI (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
s_axi_arready	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
s_axi_rdata	32	Output	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
s_axi_rresp	2	Output	Mã phản hồi đọc. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng RVALID.
s_axi_rvalid	1	Output	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
s_axi_rready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.

2 Đặc tả thanh ghi

Cửa sổ MMIO của accelerator có địa chỉ cơ sở 0x6000_0000, rộng 4 KiB (0x6000_0000 … 0x6000_0FFF). Module chỉ giải mã bit địa chỉ [11:2]. Mọi word là 32 bit; thanh ghi RW lưu đủ 32 bit, kể cả bit dự trữ, và ghi theo byte strobe. Module này sở hữu các bảng dưới đây; đặc tả hệ thống tham chiếu lại chúng.

Bảng 2.1: Bản đồ thanh ghi

Offset	Tên	Truy cập	Reset	Ghi chú
0x00	CTRL	W	0x0000_0000	Đọc ra 0. Bit lệnh tự xóa; chỉ có hiệu lực khi strb[0] = 1
0x04	STATUS	RO	0x0000_0000	DONE, BUSY, ERROR, ERR_CODE. Soft reset về 0
0x08 … 0x24	Dự trữ	—	—	Đọc 0, ghi bị bỏ qua
0x28	HW_CONFIG	RO	0x1100_2008	Hằng số phần cứng
0x2C	PERF_TOTAL	RO	0x0000_0000	Số chu kỳ BUSY
0x30	PERF_LOAD	RO	0x0000_0000	Số chu kỳ pha LOAD
0x34	PERF_COMPUTE	RO	0x0000_0000	Số chu kỳ pha COMPUTE
0x38	PERF_STORE	RO	0x0000_0000	Số chu kỳ pha STORE_C
0x3C	Dự trữ	—	—	Đọc 0, ghi bị bỏ qua
0x40	DESC_PTR	RW	0x0000_0000	Địa chỉ descriptor đầu chuỗi; soft reset giữ nguyên
0x44	ERR_DESC_ADDR	RO	0x0000_0000	Địa chỉ descriptor lỗi; FSM xóa
0x48	DESC_DONE_CNT	RO	0x0000_0000	Số descriptor STORE_C xong; FSM xóa
Mọi offset khác	Dự trữ	—	—	Đọc 0, ghi bị bỏ qua, phản hồi OKAY

QUYẾT ĐỊNH BASELINE D-01 VÀ D-03
Offset 0x08 … 0x24 dự trữ (D-01); HW_CONFIG reset 0x1100_2008 (D-03). Ghi vào thanh ghi RO hoặc offset dự trữ không có tác dụng và vẫn trả OKAY. Ký hiệu truy cập giống tài liệu hệ thống: RW đọc/ghi theo byte; RO chỉ đọc, ghi bị bỏ qua; W ghi lệnh, đọc luôn bằng 0.

CTRL (0x00)

Bảng 2.2: Các bit của CTRL

Bit	Tên	Truy cập	Mô tả
[31:3]	Dự trữ	RO	Ghi bị bỏ qua; đọc 0
[2]	START	W	Ghi 1: chấp nhận tác vụ khi rảnh, chạy chuỗi descriptor bắt đầu ở DESC_PTR. Tự xóa. Đang bận: bỏ qua, không lỗi
[1]	SOFT_RESET	W	Ghi 1: yêu cầu soft reset (phát o_soft_req). Tự xóa. Hiệu lực cả khi đang bận; thắng START trong cùng lệnh ghi
[0]	Dự trữ	RO	Ghi bị bỏ qua; đọc 0

Cả hai bit lệnh chỉ có hiệu lực khi s_axi_wstrb[0] = 1. Đọc CTRL luôn trả 0.

STATUS (0x04)

Bảng 2.3: Các bit của STATUS

Bit	Tên	Truy cập	Mô tả
[31:8]	Dự trữ	RO	Đọc 0
[7:4]	ERR_CODE	RO	Mã lỗi, hợp lệ khi ERROR = 1; bằng 0 khi không lỗi. 1: cấu hình sai; 2: lỗi AXI khi đọc (LOAD_*); 3: lỗi AXI khi ghi (STORE_C); 4: lỗi AXI khi đọc descriptor (FETCH). 5 … 15 dự trữ, không bao giờ xuất hiện
[3]	Dự trữ	RO	Đọc 0
[2]	ERROR	RO	Tác vụ kết thúc lỗi. Sticky: giữ đến tác vụ mới hoặc soft reset
[1]	BUSY	RO	Tác vụ đang chạy, hoặc đang soft reset
[0]	DONE	RO	Toàn chuỗi đã ghi xong ra DRAM. Sticky: giữ đến tác vụ mới hoặc soft reset

STATUS = {24'b0, ERR_CODE, 1'b0, ERROR, busy_flag, DONE}. Khi i_soft_busy = 1, STATUS đọc ra 0x0000_0002 (chỉ BUSY). Tại mọi thời điểm tối đa một trong BUSY, DONE, ERROR bằng 1.

HW_CONFIG (0x28)

Bảng 2.4: Các bit của HW_CONFIG

Bit	Tên	Truy cập	Giá trị	Mô tả
[31:24]	SPEC_VER	RO	0x11	Phiên bản đặc tả; 0x11 nghĩa là chỉ chuỗi descriptor và ping-pong
[23:16]	Dự trữ	RO	0x00	Đọc 0
[15:8]	MAX_DIM	RO	32	Kích thước tối đa của M, N, K
[7:0]	P	RO	8	Cạnh của array PE

Giá trị reset và giá trị đọc: {8'h11, 8'h00, 8'd32, 8'd8} = 0x1100_2008. Ghi bị bỏ qua.

> **QUYẾT ĐỊNH BASELINE D-03**
> Không có bit HAS_CHAIN và HAS_PINGPONG: chuỗi descriptor và ping-pong là chế độ duy nhất, SPEC_VER = 0x11 đã mã hóa khả năng đó. Lý do: hai bit hằng 1 không mang thông tin và tạo cách mô tả thứ hai cho cùng một việc.

PERF_TOTAL, PERF_LOAD, PERF_COMPUTE, PERF_STORE (0x2C … 0x38)

Bảng 2.5: Các bit của PERF_*

Bit	Tên	Truy cập	Mô tả
[31:0]	PERF_TOTAL (0x2C)	RO	Số chu kỳ busy_flag = 1, không tính chu kỳ chấp nhận
[31:0]	PERF_LOAD (0x30)	RO	Số chu kỳ có i_perf_load = 1 khi đang bận
[31:0]	PERF_COMPUTE (0x34)	RO	Số chu kỳ có i_perf_comp = 1 khi đang bận
[31:0]	PERF_STORE (0x38)	RO	Số chu kỳ có i_perf_store = 1 khi đang bận

Mỗi bộ đếm bão hòa ở 0xFFFF_FFFF, về 0 khi chấp nhận tác vụ, khi i_soft_clr và khi reset, giữ giá trị sau khi tác vụ kết thúc. Một chu kỳ ở 100 MHz là 10 ns, nên 0xFFFF_FFFF chu kỳ tương ứng khoảng 42,9 s.

DESC_PTR (0x40)

Bảng 2.6: Các bit của DESC_PTR

Bit	Tên	Truy cập	Mô tả
[31:0]	DESC_PTR	RW	Địa chỉ byte descriptor đầu chuỗi trong DRAM; phải căn 64 byte (DESC_PTR[5:0] = 0). Ghi theo byte strobe; đọc ra đúng giá trị đã ghi, không phải bản chụp. Ghi được mọi lúc, kể cả khi BUSY; giá trị mới có hiệu lực ở lần START được chấp nhận kế tiếp

Register File lưu đủ 32 bit và không kiểm căn lề; Main FSM kiểm DESC_PTR[5:0] $\neq$ 0 và báo ERR_CODE = 1 (D-11), không truy cập bus.

ERR_DESC_ADDR (0x44) và DESC_DONE_CNT (0x48)

Bảng 2.7: Các bit của ERR_DESC_ADDR và DESC_DONE_CNT

Bit	Tên	Truy cập	Mô tả
[31:0]	ERR_DESC_ADDR	RO	Địa chỉ DRAM của descriptor lỗi sớm nhất (hoặc DESC_PTR khi lệch căn). Trả i_err_desc_addr; hợp lệ khi ERROR = 1
[31:0]	DESC_DONE_CNT	RO	Số descriptor đã STORE_C xong với phản hồi B cuối là OKAY. Trả i_desc_done_cnt; khi ERROR bằng chỉ số descriptor lỗi f; khi DONE bằng số descriptor của chuỗi

Hai thanh ghi này không có bộ nhớ trong Register File: đọc trả trực tiếp giá trị trên cổng vào. Main FSM xóa chúng khi nhận o_start_pulse (giá trị mới thấy được từ chu kỳ sau xung, trước mọi lần đọc mà CPU có thể phát sau phản hồi B của lệnh ghi CTRL) và khi soft clear.

3 Mô tả chức năng

3.1 Tổ chức chức năng

Register File gồm hai khối con: `gemm_axil_slave` (giao thức AXI4-Lite) và `gemm_rf_regs` (giải mã, thanh ghi, trạng thái); `gemm_rf_perf` là bộ đếm hiệu năng. Module không có FSM dạng trạng thái đặt tên; trạng thái gồm các cờ `busy_flag`, DONE, ERROR và các cờ giao thức `aw_v`, `w_v`, `s_axi_bvalid`, `s_axi_rvalid`.

3.1.1 `gemm_axil_slave`

Giao thức AXI4-Lite

- Thanh ghi giữ `aw_v`/`aw_addr` và `w_v`/`w_data`/`w_strb`. `s_axi_awready` = ¬`aw_v` ∧ ¬`s_axi_bvalid`; `s_axi_wready` = ¬`w_v` ∧ ¬`s_axi_bvalid`. AW và W được nhận theo thứ tự bất kỳ
- `wr_fire` = `aw_v` ∧ `w_v` ∧ ¬`s_axi_bvalid`: lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ đó; cùng cạnh: `aw_v`, `w_v` ← 0, `s_axi_bvalid` ← 1, `s_axi_bresp` = 0
- `s_axi_bvalid` giữ tới `s_axi_bready`. Tối đa một lệnh ghi đang xử lý
- `s_axi_arready` = ¬`s_axi_rvalid`. Ở chu kỳ bắt tay AR (`rd_fire`): `s_axi_rdata` ← giá trị đọc tính từ trạng thái đầu chu kỳ, `s_axi_rvalid` ← 1; giữ tới `s_axi_rready`. Tối đa một lệnh đọc đang xử lý
- Chỉ dùng bit địa chỉ [11:2]. Đọc và ghi cùng chu kỳ được phép; đọc trả giá trị trước lệnh ghi
- Mux đọc là `case` trên `araddr[11:2]`; offset không có trong bản đồ thanh ghi trả 0

3.2 Thanh ghi

Bảng 3.1: Hành vi từng offset (`gemm_rf_regs`)

Offset	Tên	Đọc ra	Ghi	Soft reset
0x00	`CTRL`	0	Lệnh, chỉ khi `strb[0]` = 1	—
0x04	`STATUS`	{24'b0, `ERR_CODE`, 1'b0, ERROR, `busy_flag`, DONE}; khi `i_soft_busy` = 1 đọc ra 0x0000_0002 (chỉ BUSY)	Bỏ qua	Về 0
0x08 … 0x24	Dự trữ	0	Bỏ qua	—
0x28	`HW_CONFIG`	{8'h11, 8'b0, 8'd32, 8'(P)}	Bỏ qua	—
0x2C … 0x38	`PERF_*`	Bộ đếm	Bỏ qua	Về 0
0x3C	Dự trữ	0	Bỏ qua	—
0x40	`DESC_PTR`	Giá trị lưu, đủ 32 bit	Theo byte strobe	Giữ
0x44, 0x48	`ERR_DESC_ADDR`, `DESC_DONE_CNT`	`i_err_desc_addr`, `i_desc_done_cnt`	Bỏ qua	FSM xóa
Khác	Dự trữ	0	Bỏ qua	—

Bảng 3.2: Lệnh ghi CTRL có `strb[0]` = 1

Bit ghi	Hành vi ở cạnh clock của `wr_fire`
SOFT_RESET (CTRL[1]) = 1	`o_soft_req` ← 1 (xung ở chu kỳ sau). START bị bỏ qua
START (CTRL[2]) = 1, và rảnh	Chấp nhận
START, đang bận	Bỏ qua, không đổi gì
Không bit nào, hoặc chỉ CTRL[0] / CTRL[31:3]	Không có tác dụng

Chấp nhận và trạng thái

- "Rảnh" = ¬busy_flag ∧ ¬i_soft_busy ∧ ¬o_soft_req. Khi chấp nhận, ở cùng cạnh clock: o_cfg_desc_ptr ← DESC_PTR; busy_flag ← 1; DONE, ERROR, ERR_CODE ← 0; PERF_* ← 0; o_start_pulse ← 1
- i_done_pulse: DONE ← 1, busy_flag ← 0. i_err_pulse: ERROR ← 1, ERR_CODE ← i_err_code, busy_flag ← 0. BUSY về 0 đúng ở cạnh clock DONE hoặc ERROR được đặt
- Khi i_soft_busy = 1, STATUS đọc ra 0x0000_0002 (chỉ BUSY), nên tại mọi thời điểm tối đa một trong BUSY, DONE, ERROR bằng 1
- i_soft_clr: busy_flag, DONE, ERROR, ERR_CODE, PERF_* ← 0. DESC_PTR và o_cfg_desc_ptr giữ nguyên. Ưu tiên: i_soft_clr > chấp nhận > i_done_pulse/i_err_pulse
- o_start_pulse, o_cfg_desc_ptr và busy_flag = 1 có hiệu lực ở cùng chu kỳ s_axi_bvalid lên 1, nên mọi lần đọc STATUS bắt đầu sau khi CPU nhận B đều thấy tác vụ mới
- Ghi DESC_PTR khi đang BUSY được lưu nhưng không đổi o_cfg_desc_ptr; giá trị mới chỉ dùng ở lần chấp nhận kế tiếp

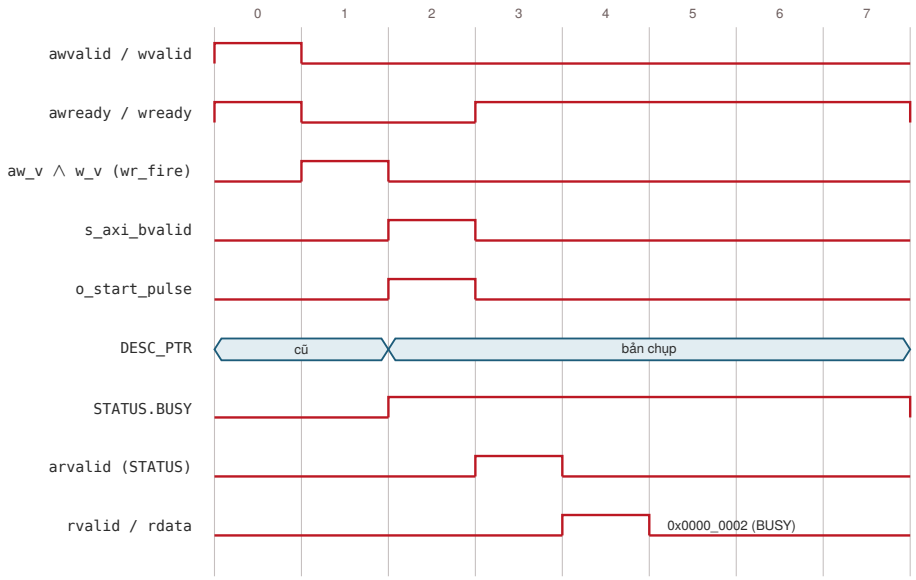
> **QUYẾT ĐỊNH BASELINE D-23**
> i_done_pulse và i_err_pulse không bao giờ cùng một chu kỳ. DONE đặt khi cả chuỗi đã ra DRAM; ERROR đặt cùng ERR_CODE một lần cho mỗi tác vụ; cả hai sticky đến tác vụ mới hoặc soft clear.

3.2.1 gemm_rf_perf

Bộ đếm hiệu năng

- Ở mỗi chu kỳ có busy_flag = 1 (không phải chu kỳ chấp nhận): PERF_TOTAL++; PERF_LOAD++ nếu i_perf_load; PERF_COMPUTE++ nếu i_perf_comp; PERF_STORE++ nếu i_perf_store. Mỗi bộ đếm bão hòa ở 0xFFFF_FFFF
- Bộ đếm về 0 khi chấp nhận tác vụ, khi i_soft_clr và khi rst_n; giữ giá trị sau khi tác vụ kết thúc
- Ba pha chồng nhau nên PERF_LOAD + PERF_COMPUTE + PERF_STORE có thể lớn hơn PERF_TOTAL

3.3 Timing



Hình 3.1: Ghi START (AW, W cùng chu kỳ 0, bready = 1) rồi đọc STATUS. Lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ 1; B, xung start và BUSY cùng xuất hiện ở chu kỳ 2. Mỗi chu kỳ là 10 ns ở 100 MHz

Bảng 3.3: Độ trễ

Sự kiện	Chu kỳ
Bắt tay AW và W đều xong ở t → `wr_fire`	t + 1 (10 ns)
`wr_fire` ở t → `s_axi_bvalid`, `o_start_pulse`, `DESC_PTR` (bản chụp), BUSY	t + 1 (10 ns)
`wr_fire` (CTRL, SOFT_RESET = 1) ở t → `o_soft_req`	t + 1 (10 ns)
Bắt tay AR ở t → `s_axi_rvalid`	t + 1 (10 ns)
`i_done_pulse` / `i_err_pulse` ở t → STATUS mới	t + 1 (10 ns)
Thông lượng ghi liên tục	1 lệnh mỗi 3 chu kỳ (30 ns) khi `bready` = 1
Thông lượng đọc liên tục	1 lệnh mỗi 2 chu kỳ (20 ns) khi `rready` = 1

QUYẾT ĐỊNH BASELINE D-20
Chu kỳ danh định 10,000 ns (100 MHz). Mọi thời gian ns trong tài liệu quy đổi theo 10 ns/chu kỳ. Thông lượng đọc 1 lệnh mỗi 2 chu kỳ suy ra từ `s_axi_arready` = ¬`s_axi_rvalid`: bắt tay AR ở t, `s_axi_rvalid` = 1 ở t + 1, nhận R ở t + 1, `s_axi_arready` = 1 trở lại ở t + 2.

3.4 Reset, hiện thực, kiểm chứng

Reset

- `rst_n_sync` = 0: mọi thanh ghi về giá trị reset trong bản đồ thanh ghi (RW = 0, RO = 0, `HW_CONFIG` là hằng); `aw_v`, `w_v`, `s_axi_bvalid`, `s_axi_rvalid`, `busy_flag`, DONE, ERROR, `o_start_pulse`, `o_soft_req` về 0; `o_cfg_desc_ptr` về 0
- Flip-flop trạng thái dùng `always_ff@(posedgeclkornegedgerst_n_sync)`
- `i_soft_clr`: `busy_flag`, DONE, ERROR, `ERR_CODE`, `PERF_*` ← 0. `DESC_PTR` và `o_cfg_desc_ptr` giữ nguyên. Ưu tiên: `i_soft_clr` > chấp nhận > `i_done_pulse`/`i_err_pulse`. Giao dịch AXI4-Lite đang dở không bị ảnh hưởng
- Sau soft clear: STATUS = 0, không báo DONE, không báo ERROR

Hiện thực

- SystemVerilog tổng hợp được, không latch. Mux đọc là `case` trên `araddr[11:2]`
- Mọi port ra lấy trực tiếp từ flip-flop, trừ `s_axi_awready`, `s_axi_wready`, `s_axi_arready`, vốn là hàm tổ hợp của cờ trạng thái nội bộ; không có đường tổ hợp từ input nào tới output nào
- Không có bộ nhân, không có RAM; tài nguyên: bốn bộ đếm 32 bit bão hòa, một thanh ghi 32 bit `DESC_PTR` và một bản chụp 32 bit

Assertion bắt buộc (SVA)

Bảng 3.4: Danh sách assertion của `gemm_register_file`

Mã	Khẳng định
AS-01	`s_axi_bvalid` ∧ ¬`s_axi_bready` → `s_axi_bvalid` ở chu kỳ sau
AS-02	`s_axi_rvalid` ∧ ¬`s_axi_rready` → `s_axi_rvalid` ở chu kỳ sau và `s_axi_rdata` không đổi
AS-03	`o_start_pulse` → ¬`busy_flag` ở chu kỳ trước
AS-04	Tối đa một trong STATUS[0], STATUS[1], STATUS[2] bằng 1
AS-05	`busy_flag` → `stable(o_cfg_desc_ptr)`
AS-06	`i_done_pulse` ∨ `i_err_pulse` → `busy_flag`
AS-07	`s_axi_bresp` = 2'b00 khi `s_axi_bvalid`; `s_axi_rresp` = 2'b00 khi `s_axi_rvalid`
AS-08	¬(`o_start_pulse` ∧ `o_soft_req`)

Bảng 3.4: Danh sách assertion của `gemm_register_file` (tiếp theo)

Mã	Khẳng định
AS-09	STATUS[7:4] $\leq$ 4 mọi lúc; STATUS[7:4] $\neq$ 0 $\rightarrow$ STATUS[2] = 1
AS-10	Đọc offset dự trữ (0x08 … 0x24, 0x3C và mọi offset không liệt kê) trả `s_axi_rdata` = 0

Kiểm tra chức năng

- Đọc lại `DESC_PTR` đúng từng byte với strobe ngẫu nhiên; ghi vào RO và offset dự trữ không đổi gì và trả OKAY
- Đọc `HW_CONFIG` = 0x1100_2008 sau reset
- START khi bận bị bỏ qua; SOFT_RESET khi bận được nhận; ghi cả hai bit trong một lệnh chỉ phát `o_soft_req`
- Chụp `o_cfg_desc_ptr` và phát `o_start_pulse` trước khi trả B; ghi `DESC_PTR` khi bận không đổi `o_cfg_desc_ptr`
- PERF bão hòa ở 0xFFFF_FFFF (nạp giá trị gần đầy bằng ép tín hiệu trong mô phỏng) và về 0 khi chấp nhận

KIỂM CHỨNG BẰNG MÔ HÌNH

(Mô hình nằm ở `gemm-accel/model/`, đã chạy lại với baseline ngày 12/11/2026; kết quả đạt.) `gemm_regfile_model.py`: 200 lượt $\times$ 60 lệnh ngẫu nhiên (AW và W lệch nhau 0 … 3 chu kỳ, `bready` ngẫu nhiên, byte strobe ngẫu nhiên, offset dự trữ, START/SOFT_RESET xen kẽ với FSM giả lập kết thúc tác vụ sau thời gian ngẫu nhiên): đọc lại thanh ghi RW đúng từng byte, offset dự trữ đọc 0, STATUS không bao giờ có hai cờ cùng lúc; 300 lượt riêng kiểm Register File phát `start_pulse` và chụp cấu hình trước khi trả phản hồi B cho lệnh ghi CTRL, và mọi lần đọc STATUS sau khi lệnh ghi hoàn tất đều thấy tác vụ mới.